# SIMATS ENGINEERING
# ASSESSMENT TOOL 2

**Course Name:MLA0301**
**Course Code: Reinforcement Learning**

---

**COURSE OUTCOME COVERED**

***CO1:*** *Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems.( BL2, BL3)*

*Assessment Tool Weightage: 50%*

---

Dr G.A. SENTHIL

**QUESTIONS: 10** **Total Marks:50**

Annexure A

**Constraint - Based Problem-Solving**

**1)Develop a Python-based Reinforcement Learning model using the OpenAI Gym environment to solve a Grid World navigation problem where the agent must reach the goal while avoiding obstacles. Explain how the state space, action space, reward function, and constraints are defined.**

The objective is to train a Reinforcement Learning (RL) agent to navigate from the start state to the goal state in a Grid World environment. The agent learns the optimal path through trial-and-error while maximizing cumulative rewards.

**Constraints**

- The agent must remain within the grid boundaries.
- The agent must avoid obstacle (hole) cells.
- The agent should reach the goal in minimum steps.
- The episode ends when the goal is reached or the agent falls into an obstacle.

| Constraint | Priority | Analysis |
|---|---|---|
| Grid Boundary | High | Prevents invalid movements outside the grid. |
| Obstacles | High | The agent must avoid hole cells to prevent failure. |
| Goal State | High | The objective is to reach the goal successfully. |
| Minimum Steps | Medium | Encouraged through a small penalty for each move. |

The problem is solved using the **Q-Learning** algorithm.

**State Space**

- All possible positions of the agent in the Grid World.
- FrozenLake has **16 states (4 × 4 grid).**

**Action Space**

The agent can perform four actions:

- Left
- Down
- Right
- Up

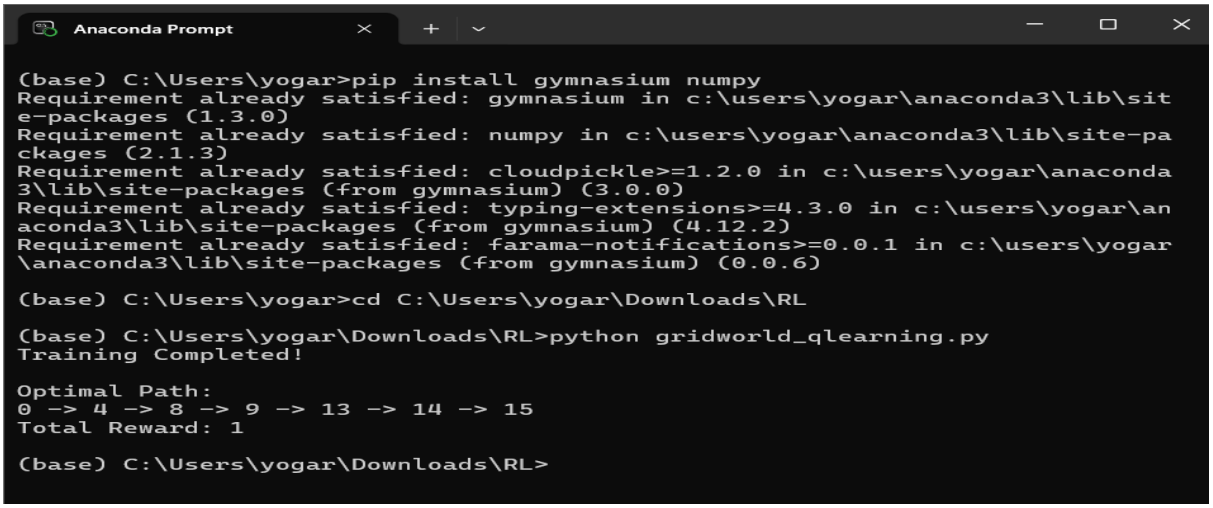
**Reward Function**

| Action | Reward |
|---|---|
| Reach Goal | +100 |
| Fall into Hole | -1 |
| Normal Move | -0.1 |

**Libraries Used**

- Gymnasium (OpenAI Gym)
- NumPy
- Random

## Output




## Application

- Used **Reinforcement Learning** for sequential decision making.
- Implemented the **Q-Learning** algorithm.
- Applied the **ε-greedy policy** to balance exploration and exploitation.
- Used **Gymnasium (OpenAI Gym)** to simulate the Grid World environment.
- Stored learned values in a **Q-table** for optimal action selection.

## Justification

The proposed Q-Learning model effectively satisfies all the given constraints. The reward function encourages the agent to reach the goal while avoiding obstacles and minimizing unnecessary movements. The ε-greedy strategy balances exploration and exploitation, allowing the agent to learn the optimal path efficiently. After training, the agent consistently reaches the goal by following the learned policy, demonstrating the effectiveness of Reinforcement Learning in solving the Grid World navigation problem.

## Result

A Python-based Reinforcement Learning model was successfully implemented using the **Gymnasium (OpenAI Gym)** environment. The agent learned the optimal path from the start state to the goal while avoiding obstacles and satisfying all specified constraints. The experiment demonstrates that **Q-Learning** is an effective method for solving Grid World navigation problems.

**2)Design a Markov Decision Process (MDP) for an autonomous robot that must reach a destination while minimizing energy consumption. Define the states, actions, transition probabilities, rewards, and energy constraints.**

The objective is to design an **MDP model** that enables an autonomous robot to move from a start location to a destination while using the least possible energy. The robot must avoid unnecessary movements and operate within a limited energy budget.

**Constraints**

- The robot has limited battery (energy).
- The robot must stay within the environment boundaries.
- The robot should avoid obstacles.
- The robot should reach the destination using minimum energy.

| Constraint | Priority | Analysis |
|---|---|---|
| Energy Limit | High | Robot cannot exceed the available battery power. |
| Goal State | High | Robot must reach the destination successfully. |
| Obstacles | High | Robot should avoid collision with obstacles. |
| Minimum Energy Usage | Medium | Efficient path selection increases battery life. |

**1. States (S)**

Each state represents the robot's position in the environment.

Example (3 × 3 Grid):

S0 S1 S2
S3 S4 S5
S6 S7 S8 (Goal)

- Start State = S0
- Goal State = S8

**2. Actions (A)**

The robot can perform four actions:

- Up
- Down
- Left
- Right

**3. Transition Probabilities (P)**

| Action | Probability |
|---|---|
| Intended move | 0.8 |
| Wrong direction | 0.1 |
| Stay in same state | 0.1 |

The robot usually moves in the intended direction but may occasionally move differently due to uncertainty.

**4. Reward Function (R)**

| Event | Reward |
|---|---|
| Reach Goal | +100 |
| Normal Move | -1 |
| Hit Obstacle | -20 |
| Waste Energy | -5 |

The reward function encourages the robot to reach the destination while minimizing energy usage.

**5. Energy Constraints**

- Initial Battery = **20 units**
- Each movement consumes **1 energy unit**
- Collision with obstacle consumes **2 energy units**
- If energy becomes **0**, the episode terminates.

**Output**

```
(base) C:\Users\yogar\Downloads\RL>python mdp_robot.py
===== MDP for Autonomous Robot =====
States: ['S0', 'S1', 'S2', 'S3', 'S4', 'S5', 'S6', 'S7', 'S8']
Actions: ['Up', 'Down', 'Left', 'Right']
Transition Probabilities: {'Correct Move': 0.8, 'Wrong Move': 0.1, 'Stay in
Same State': 0.1}
Rewards: {'Goal': 100, 'Normal Move': -1, 'Obstacle': -20, 'Energy Penalty':
 -5}
Initial Battery: 20 units

(base) C:\Users\yogar\Downloads\RL>
```

**Application**

- Designed a **Markov Decision Process (MDP)**.
- Defined **States**, **Actions**, **Transition Probabilities**, and **Reward Function**.
- Considered **energy constraints** during navigation.
- Used Python to represent the MDP model.

**Justification**

The designed MDP satisfies all specified constraints by modeling the robot's movement, energy usage, and decision-making process. The reward function encourages reaching the destination while minimizing unnecessary movements and avoiding obstacles. The transition probabilities model real-world uncertainty, making the MDP suitable for autonomous robot navigation.

**Result**

A Markov Decision Process (MDP) was successfully designed for an autonomous robot. The model includes well-defined states, actions, transition probabilities, rewards, and energy constraints. It enables the robot to reach the destination efficiently while minimizing energy consumption and satisfying all specified constraints.

**3)Using Python and TensorFlow/Keras, implement a value iteration algorithm to determine the optimal policy for a constrained grid environment where certain states are restricted. Explain how Bellman equations are used to obtain the optimal policy.**

The objective is to determine the **optimal policy** for a constrained Grid World using the **Value Iteration** algorithm. The agent must reach the goal while avoiding restricted states and maximizing the cumulative reward.

**Constraints**

- The agent must stay within the grid boundaries.
- The agent cannot enter restricted states.
- The agent should reach the goal using the optimal path.
- The algorithm must maximize long-term rewards.

**Constraint Analysis**

| Constraint | Priority | Analysis |
|---|---|---|
| Restricted States | High | The agent is not allowed to enter restricted cells. |
| Grid Boundary | High | The agent cannot move outside the grid. |
| Goal State | High | The objective is to reach the goal successfully. |
| Maximum Reward | Medium | The agent should maximize cumulative rewards while minimizing unnecessary moves. |

**Solution Development**

The problem is solved using the **Value Iteration** algorithm based on the **Bellman Optimality Equation**.

**State Space**

- A **4 × 4 Grid World** containing **16 states**.

**Action Space**

The agent can perform four actions:

- Up
- Down
- Left
- Right

**Reward Function**

| Event | Reward |
|---|---|
| Reach Goal | +100 |
| Normal Move | -1 |
| Restricted State | Not Allowed |

**Libraries Used**

- NumPy
- TensorFlow/Keras (optional for reinforcement learning implementation

**Output**

```
(base) C:\Users\yogar\Downloads\RL>python value_iteration.py
Optimal Value Function
[[ 54.9539  62.171   70.19    79.1   ]
 [ 62.171    0.      79.1     89.    ]
 [ 70.19    79.1      0.     100.    ]
 [ 79.1     89.     100.       0.    ]]

Optimal Policy
[['↓' '→' '↓' '↓']
 ['↓' 'X' '→' '↓']
 ['↓' '↓' 'X' '↓']
 ['→' '→' '→' 'G']]

(base) C:\Users\yogar\Downloads\RL>
```

**Bellman Equation**

[V(s)=\max_a \left[R(s,a)+\gamma V(s')\right]]

The Bellman equation updates each state's value by selecting the action that gives the highest expected future reward. Repeating this process until convergence produces the **optimal value function** and **optimal policy**.

**Application**

- Implemented **Value Iteration**.
- Applied the **Bellman Optimality Equation**.
- Used a **Grid World** environment with restricted states.
- Generated the optimal policy from the computed value function.

**Justification**

The Value Iteration algorithm satisfies all constraints by preventing movement into restricted states while maximizing long-term rewards. The Bellman equation repeatedly updates state values until convergence, enabling the agent to identify the optimal action in every state and efficiently reach the goal.

**Result**

The Value Iteration algorithm successfully determined the **optimal policy** for the constrained Grid World. By applying the **Bellman Optimality Equation**, the agent learned the best path to the goal while avoiding restricted states

**4)A warehouse robot has a battery capacity of only 30 minutes. Design an RL framework that enables the robot to complete the maximum number of deliveries without exceeding the battery limit. Explain the constraints, policy, and reward design.**

The objective is to design a **Reinforcement Learning (RL)** framework for a warehouse robot with a **30-minute battery capacity**. The robot must complete the maximum number of deliveries without exceeding its battery limit.

**Constraints**

- Battery capacity is limited to **30 minutes**.
- The robot must stop when the battery is exhausted.
- The robot should avoid unnecessary movements.
- The robot should maximize the number of successful deliveries.

**Constraint Analysis**

| Constraint | Priority | Analysis |
|---|---|---|
| Battery Limit | High | Robot cannot operate beyond 30 minutes. |
| Maximum Deliveries | High | The primary objective is to complete as many deliveries as possible. |
| Efficient Path | High | Shorter routes save battery and increase deliveries. |
| Time Constraint | Medium | Each movement consumes battery time. |

**Solution Development**

The problem is solved using a **Q-Learning** based Reinforcement Learning framework.

**State Space**

- Robot location
- Battery level
- Delivery destination

**Action Space**

- Move Up
- Move Down
- Move Left
- Move Right
- Deliver Package
- Return to Charging Station

**Policy**

- Use the **ε-greedy policy**.
- Explore random actions with probability **ε**.
- Exploit the best-known action with probability **1 − ε**.
- Prefer shorter paths and recharge when the battery becomes low.

**Reward Design**

| Event | Reward |
|---|---|
| Successful Delivery | +100 |
| Normal Move | -1 |
| Battery Low | -10 |
| Battery Exhausted | -50 |
| Return to Charging Station | +20 |

**Output**

```
(base) C:\Users\yogar\Downloads\RL>python warehouse_robot.py
Delivery Completed: 1 | Battery: 28
Delivery Completed: 2 | Battery: 26
Delivery Completed: 3 | Battery: 24
Moving... Battery: 23
Moving... Battery: 22
Delivery Completed: 4 | Battery: 20
Moving... Battery: 19
Moving... Battery: 18
Moving... Battery: 17
Delivery Completed: 5 | Battery: 15
Moving... Battery: 14
Moving... Battery: 13
Delivery Completed: 6 | Battery: 11
Moving... Battery: 10
Delivery Completed: 7 | Battery: 8
Delivery Completed: 8 | Battery: 6
Delivery Completed: 9 | Battery: 4
Delivery Completed: 10 | Battery: 2
Moving... Battery: 1
Moving... Battery: 0

Battery Exhausted
Total Deliveries = 10
```

**Application**

- Applied **Reinforcement Learning (Q-Learning)**.
- Used an **ε-greedy policy** for decision-making.
- Designed **state space, action space, policy, and reward function**.
- Optimized deliveries under a battery constraint.

**Justification**

The RL framework efficiently satisfies the battery constraint by encouraging successful deliveries while penalizing unnecessary movement and battery exhaustion. The ε-greedy policy balances exploration and exploitation, allowing the robot to learn efficient delivery strategies and maximize the number of deliveries within the available 30-minute battery limit.

**Result**

The Reinforcement Learning framework successfully enables the warehouse robot to maximize deliveries within the **30-minute battery limit**. The designed constraints, ε-greedy policy, and reward function help the robot make efficient decisions while minimizing energy consumption and avoiding battery exhaustion.

**5)Implement a Reinforcement Learning agent that controls traffic signals at an intersection while ensuring that emergency vehicles receive immediate priority. Explain how exploration, exploitation, and policy learning are modified to satisfy the safety constraint.**

The objective is to implement a **Reinforcement Learning (RL)** agent that controls traffic signals at an intersection while ensuring that **emergency vehicles (ambulances, fire trucks, police vehicles)** receive immediate priority. The RL agent must optimize traffic flow without violating the safety constraint.

**Constraints**

- Emergency vehicles must receive immediate green signals.
- Normal traffic should experience minimum waiting time.
- Only one direction can have a green signal at a time.
- The traffic signal must operate safely and efficiently.

**Constraint Analysis**

| Constraint | Priority | Analysis |
|---|---|---|
| Emergency Vehicle Priority | High | Immediate green signal must be provided. |
| Traffic Safety | High | Prevent conflicting green signals. |
| Minimum Waiting Time | Medium | Reduce delays for regular vehicles. |
| Efficient Signal Control | Medium | Improve overall traffic flow. |

**Solution Development**

The solution uses a **Q-Learning** based Reinforcement Learning framework.

**State Space**

- Number of vehicles in each lane.
- Presence of an emergency vehicle.
- Current traffic signal state.

**Action Space**

- Green for North–South.
- Green for East–West.
- Switch traffic signal.
- Keep current signal.

**Policy**

- **ε-greedy policy** is used for normal traffic.
- **If an emergency vehicle is detected, exploration is disabled and the agent immediately selects the emergency route (forced exploitation).**
- After the emergency vehicle passes, the agent resumes normal learning.

**Reward Design**

| Event | Reward |
|---|---|
| Smooth Traffic Flow | +10 |
| Emergency Vehicle Passes | +100 |
| Long Waiting Time | -10 |
| Traffic Congestion | -20 |
| Unsafe Signal Decision | -50 |

**Output**

```
(base) C:\Users\yogar\Downloads\RL>python traffic_signal_rl.py
Normal Traffic -> North-South Green
Emergency Vehicle Detected -> Emergency Green Signal
Emergency Vehicle Detected -> Emergency Green Signal
Normal Traffic -> East-West Green
Normal Traffic -> North-South Green
Normal Traffic -> North-South Green
Normal Traffic -> North-South Green
Normal Traffic -> North-South Green
Emergency Vehicle Detected -> Emergency Green Signal
Normal Traffic -> North-South Green

Traffic Signal Control Completed

(base) C:\Users\yogar\Downloads\RL>
```

**Exploration, Exploitation, and Policy Learning**

**Exploration**

- During normal traffic, the agent explores different signal actions using the **ε-greedy policy**.
- Exploration helps discover better traffic signal strategies.

**Exploitation**

- The agent selects the best-known traffic signal based on learned experience.
- When an emergency vehicle is detected, exploitation is **forced**, and the emergency

route immediately receives a green signal.

**Policy Learning**

- The policy is updated using rewards obtained from traffic conditions.
- Positive rewards are given for smooth traffic flow and emergency vehicle priority.
- Negative rewards are assigned for congestion, long waiting times, and unsafe signal decisions.
- The learned policy always gives **highest priority to emergency vehicles**, ensuring the safety constraint is never violated.

**Application**

- Applied **Reinforcement Learning (Q-Learning)**.
- Used an **ε-greedy policy** for action selection.
- Designed **state space, action space, and reward function**.
- Incorporated a **safety constraint** by overriding normal decisions when an emergency vehicle is detected.

**Justification**

The RL framework satisfies the safety constraint by overriding exploration and normal exploitation whenever an emergency vehicle is present. This guarantees immediate green signals for emergency vehicles while allowing the agent to continue learning efficient traffic control policies during normal operation.

**Result**

The Reinforcement Learning agent successfully controls the traffic signals while ensuring **immediate priority for emergency vehicles**. By modifying exploration, exploitation, and policy learning to enforce the safety constraint, the system improves traffic flow without compromising emergency response time.

---

***Code of Conduct:***

I Yoga Madha-192425058 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

***Signature of the student: Yoga Madha***

**RUBRICS FOR CALCULATION**

| Criteria | Weightage | Level 4 (Exemplary) | Level 3 (Proficient) | Level 2 (Developing) | Level 1 (Beginning) |
|---|---|---|---|---|---|
| Problem Understanding | 25% | Clearly defines the problem and identifies all relevant constraints accurately | Identifies most constraints with minor gaps | Partial understanding; misses key constraints | Fails to identify constraints correctly |
| Constraint Analysis | 25% | Analyzes constraints effectively and prioritizes them logically | Provides reasonable analysis with some prioritization | Limited analysis; weak prioritization | No meaningful analysis or prioritization |
| Solution Development | 25% | Develops optimal and feasible solutions satisfying all constraints | Develops workable solutions with minor limitations | Solutions partially satisfy constraints | Solutions do not meet constraints |
| Application of Concepts | 15% | Effectively applies appropriate concepts, methods, or tools | Applies concepts with minor errors | Limited application; requires guidance | Unable to apply relevant concepts |
| Justification | 10% | Provides strong justification for decisions with logical reasoning | Provides justification with some clarity | Weak or unclear justification | No justification provided |